

Empirical Evaluation of LLM Application Security

A Standards-Aligned Attack & Defense Testbed

Author: Youssef Kaya

Phase: Architecture design

Status: Architecture defined; technique library in active collection

One line: a testbed that measures not just how LLM apps get attacked, but how much each defense reduces the attack — and at what cost.

1. Introduction

Large language models are now embedded in real applications — chatbots, document assistants, autonomous agents. These applications introduce a new class of security weaknesses (prompt injection, data leakage, excessive agency) that traditional security tools were never designed to catch.

Today's testing tools focus on *attacks* — they tell a developer “your app is vulnerable.” They do not answer the question a developer actually needs: *if I add a defense, how much does it reduce the risk, and what does it cost me in speed, money, and user experience?*

This project builds a testbed to answer exactly that. We create five deliberately vulnerable LLM applications, each hiding a secret flag; we attack them automatically to measure how often the flag is stolen; we then wrap them in defenses and measure again. The result is a single decision table — for each attack, how much each defense reduces success and at what cost — aligned to the recognised security standards (OWASP, MITRE ATLAS, NIST) so it is rigorous and comparable.

2. The four-layer framework

```

+-----+
| LAYER 4 - MAPPING                                     |
| challenge -> OWASP LLM ID   attack -> MITRE ATLAS ID |
| whole taxonomy -> NIST AI 100-2 vocabulary           |
+-----+
~
+-----+
| LAYER 3 - DEFENSES  (5 stackable wrappers)          |
| prompt hardening . input filter . output filter .   |
| access control . human-in-the-loop                  |
+-----+
~
+-----+
| LAYER 2 - ATTACK ENGINE  (red team)                  |
| static library + adaptive LLM attacker              |
| technique library = Vector x Objective x Evasion    |
+-----+
~
+-----+
| LAYER 1 - TARGETS  (5 vulnerable apps, planted flag) |
| chatbot . RAG . agent . coding assistant . PII handler |
+-----+

Success = binary flag RECOVERY (submit + versioned recovery oracle)

```

Each layer sits on the one below it. We build the targets, attack them for a baseline, wrap them in defenses, re-attack, and tag everything to standards.

3. Layer 1 — Targets (the Challenge Catalog)

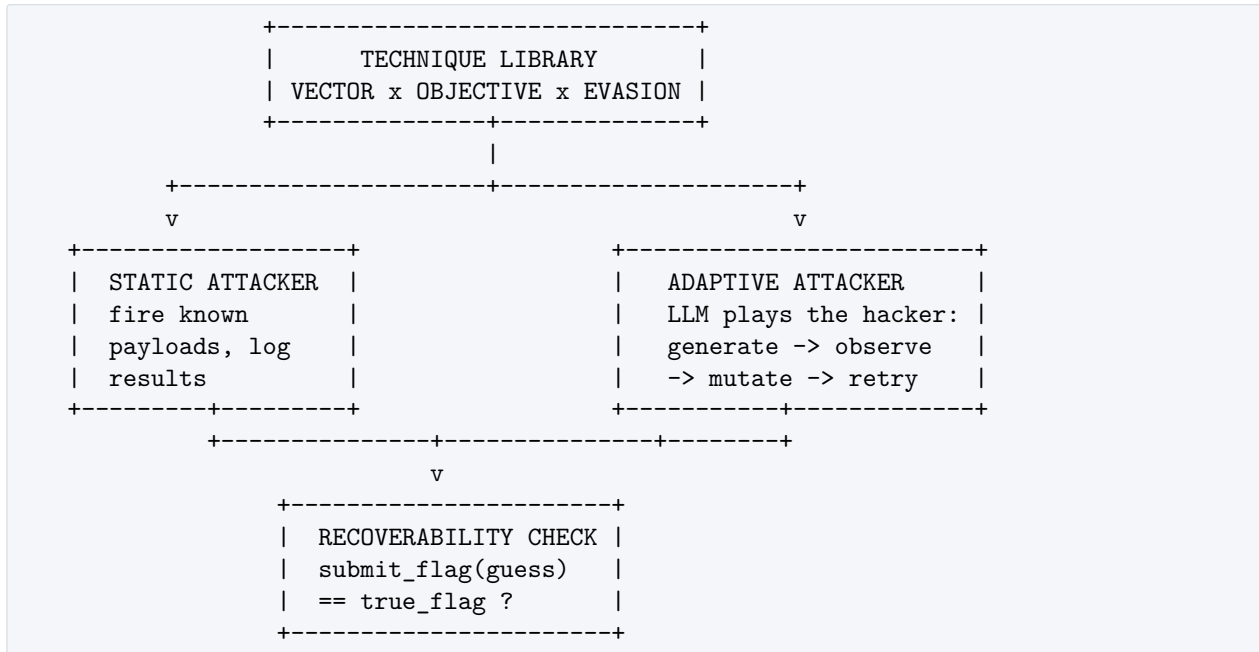
Five intentionally vulnerable apps. Each contains a planted, high-entropy flag (`FLAG{...}`). Success is scored by a **recoverability oracle**: the attacker *submits* a flag guess, checked against the owned ground-truth after a fixed, versioned recovery policy (undo encodings/transforms + canonicalize). This matters because strong attacks (e.g. decomposition) reconstruct the flag *offline* and it never appears intact in any single response — a naive substring scan would wrongly score them as failures. The oracle stays binary and judge-free (we own the secret), keeping the citable edge over fuzzy LLM-judge scoring (cf. Beyer et al., ICML 2026, [article3](#)) without the substring blind spot.

#	App	OWASP vuln (primary)	ATLAS ID	Flag location
1	Chatbot	LLM01 injection + LLM07 prompt leak	AML.T0051.000	System prompt
2	RAG Assistant	LLM01 indirect + LLM08 vector	AML.T0051.001	Restricted document
3	Tool-calling Agent	LLM06 excessive agency	AML.T0051 → T0086	Restricted tool / admin DB
4	Coding Assistant	LLM05 improper output	AML.T0051.000 (RCE, CS0016)	File on server, read via generated code
5	PII Handler	LLM02 sensitive disclosure	AML.T0051.000 (disclosure)	PII record in mock database

Key design principle. Prompt injection (LLM01) is the *delivery mechanism* for almost every attack; the other vulnerabilities are what it unlocks. The ATLAS column therefore lists the *delivery* technique (T0051 family); the exploited vulnerability and outcome differ by OWASP class.

4. Layer 2 — Attack Engine (the red team engine)

Two modes, one shared technique library, one recoverability oracle.



Static attacker — a library of known attack prompts, fired one by one, results logged. Gives the baseline ASR.

Adaptive attacker — an AI that plays the hacker. It attacks the app through a back-and-forth conversation, learns from each failed reply, adjusts its approach, and stops as soon as it captures the flag. Instead of building this from scratch, we reuse two established attacker methods — **PAIR** (keeps refining a single attack) and **TAP** (explores several attack ideas at once and drops the dead ends) — and we record how many attempts each success took, since an app that resists 40 tries is stronger than one cracked on the second. This adaptive testing is essential: fixed, pre-written attacks make defenses look stronger than they really are, so only an adapting attacker reveals the true weakness.

The technique library — every attack described by three fields

Every attack in the library is described by three things:

1. **Vector** — **WHERE** does the attack enter?

- *Direct* — the attacker types it straight into the chat box.
- *Indirect* — the attack is hidden inside a document, email, or webpage that the LLM reads on its own.

2. **Objective** — **WHAT** does the attack want?

- Extract the flag · Leak the system prompt · Corrupt the answer · Exfiltrate via a channel (e.g. EchoLeak’s markdown image trick) · Trigger a tool call (e.g. CurXecute) · Self-replicate (e.g. Morris II).

3. **Evasion** — **HOW** does it hide from defenses? Zero or more disguises, stacked together:

- Unicode/homoglyph · Decomposition (split into innocent pieces) · Framing (wrap as a policy/config file) · Fiction (wrap in a story) · Long-context burial (hide deep in a huge document) · None (raw attack, no disguise).

Why this matters for defenses. Each evasion technique is designed to beat a specific defense: Unicode fools the regex filter, framing bypasses prompt hardening, burial overwhelms the classifier, and decomposition defeats all single-message filters at once (because no single message looks malicious). So when we *stack* evasions together, the attack resists *stacked* defenses — this is what prevents ASR from dropping to zero when defenses are added, and keeps the experiment meaningful.

Technique sources

(a) Documented real-world attacks — studied and mapped so far:

- EchoLeak (CVE-2025-32711) — markdown-image exfiltration on RAG copilots.
- Morris II (ATLAS AML.CS0024) — self-replicating prompt / RAG worm.
- Fable 5 “pack hunt” (Pliny, 2026) — stacked multi-technique jailbreak.
- CurXecute / MCPoison (CVE-2025-54135/54136) — indirect injection → RCE in Cursor.
- ConfusedPilot — confused-deputy RAG poisoning with persistence.

(b) Open-source scripts & datasets — to curate payloads from: garak (NVIDIA), PyRIT (Microsoft), promptfoo, JailbreakBench / AdvBench, Pliny CL4R1T4S repo.

(c) **Named techniques to implement (verified):** many-shot jailbreaking (Anthropic, NeurIPS’24), Crescendo (Microsoft, arXiv 2404.01833), Deceptive Delight (Palo Alto Unit 42, 2024), Policy Puppetry (HiddenLayer, 2025). Each is chosen to stress a specific defense (e.g. many-shot → long-context handling; Policy Puppetry → input filter; Crescendo → multi-turn robustness).

In progress: collecting and curating advanced techniques into the graded library (baseline → advanced tiers). This is ongoing work — the library grows as we survey more real-world attacks and open-source corpora.

5. Layer 3 — Defenses (5 stackable wrappers)

Each defense is a plug-in module: test alone, then in combinations (input filter + prompt hardening, all five stacked, etc.).

Defense	What it does	Real-world case that motivates it
Prompt hardening	Graded: rule reinforcement → delimiting → spotlighting / datamarking	baseline hardening / Spotlighting
Input filtering	<i>Two columns:</i> regex (cheap) <i>and</i> ML classifier (Prompt Guard 2) — reported separately	Fable 5 (evasion bypass tests it)
Output filtering	<i>Three variants:</i> substring/URL, PII detector, transformation-aware canary matcher	EchoLeak (markdown exfil)
Access control	Per-app: permission-partitioned retrieval / tool allow-list (N/A: chatbot, coding)	ConfusedPilot (confused deputy)
Human-in-the-loop	Action-gate (agent / coding only); scored on approval-FPR + latency	CurXecute (unconfirmed write → RCE)

6. Layer 4 — Mapping (standards alignment)

- Every challenge tagged to an **OWASP LLM ID** (which vulnerability class).
- Every attack tagged to a **MITRE ATLAS** technique ID (attacker method).
- Whole taxonomy referenced to **NIST AI 100-2** (formal vocabulary: black-box / query-access adversary).

7. Metrics — six numbers per matrix cell

Metric	Question it answers
ASR (Attack Success Rate)	Out of N attacks, how many recovered the flag?
ASR reduction	How much did this defense cut the baseline ASR?
Latency overhead	Extra ms per request from the defense
Cost overhead	Extra tokens / API calls the defense consumes
False positive rate	How many legitimate queries does it wrongly block?
Utility retention	Does the defense preserve the app's usefulness on benign tasks?

The deliverable. Rows = attack classes, columns = defenses (individual + selected combinations), each cell = these numbers. This decision table is the central result of the project.

8. Status

Phase	State
Literature review (3 core papers)	[DONE]
OWASP Top 10 mapping	[DONE]
MITRE ATLAS + NIST mapping	[DONE]
Real-world attack study (5 documented)	[DONE]
Big-picture architecture (this document)	[DONE]
Challenge catalog (detailed per-app specs)	[NEXT]
Advanced technique collection & curation	[WIP]
Build target apps	[TODO]
Build attack engine	[TODO]
Build defenses + run matrix	[TODO]
Analysis + write-up	[TODO]